

Ω -Architect 用户指南

面向 AI4Math 的端到端形式化证明系统 — 安装、配置与接入

Ω -Architect Team

2026-06-10

目录

第一章	Ω-Architect 用户指南	3
1.1	面向 AI4Math 的端到端形式化证明系统	3
第二章	项目概述与定位	4
2.1	技术定位	4
2.2	核心设计原则	4
2.3	与现有系统的关系	4
第三章	系统架构与核心模块	5
3.1	目录结构	5
3.2	核心数据流	6
3.3	关键接口	7
3.4	验证层架构	7
第四章	技术性能指标	9
4.1	测试覆盖	9
4.2	编译性能	9
4.3	模型定价与路由	9
4.4	MiniF2F 基准进展	10
第五章	用户 Quick Start 与进阶管线	11
5.1	快速开始 (5 分钟)	11
5.2	环境要求	11
5.3	进阶开发管线	12
5.3.1	使用本地 Goedel-Prover-V2	12
5.3.2	运行基准测试	12
5.4	编程接口	12
5.5	OmegaRunner 长时间运行	13
第六章	数据集、国内外对比与交叉领域	14
6.1	AI4Math 证明题数据集对比	14
6.2	国内外相关研究工作	14

6.2.1	国际	14
6.2.2	国内	15
6.2.3	性能对比 (MiniF2F Pass@32)	15
6.3	交叉领域：物理、量子光学到形式化数学	15
6.3.1	物理数学形式化的现状	15
6.3.2	SciLean 项目	16
6.3.3	Physlib：物理形式化的社区基座	16
6.3.4	Ω-Architect 在交叉领域的定位	16
6.3.5	量子光学的特殊需求	17
6.3.6	已有开源工作	17
第七章	目标场景、开发状态与路线图	19
7.1	目标场景清单	19
7.2	已就绪能力	19
7.3	待完成	20
7.4	路线图	20
7.4.1	短期 (2026 Q3)	20
7.4.2	中期 (2026 Q4)	20
7.4.3	长期 (2027)	21
Appendices		22
附录 A	验证实例：Omega 端到端证明	22
A.1	命题	22
A.2	Omega 自动生成的 Lean 4 代码	22
A.2.1	逐行解析	22
A.3	编译结果	24
附录 B	MiniF2F 涵盖数据集	25
附录 C	技术栈依赖	26
附录 D	元数据标准与依赖图	27
D.1	章节元数据字段	27
D.2	章节依赖图	27
D.3	版本锚点注入	27
附录 E	版本历史	28

第 1 章 Ω -Architect 用户指南

§1.1 面向 AI4Math 的端到端形式化证明系统

摘要： Ω -Architect 是一个基于 Lean 4 + Mathlib 的开源形式化证明系统，面向 AI4Math 算法组、数学物理方程研究组、量子光学实验物理组。本文档提供系统架构概览、性能指标、快速入门指南、用户接入说明、国内外研究对比，以及物理/量子领域到形式化数学的交叉前景。

文档版本	v0.1.0 (commit \texttt{2953191})
提交日期	2026-06-10
repo 版本	v0.1.0 (与 pyproject.toml 同步)
许可证	Apache 2.0
目标	MiniF2F 244 题通过率 >95%
当前开发集	Easy 3/3 , Medium 1/4, Hard 0/3

阅读指引：

读者群体	推荐章节
AI4Math 算法组	ch04 Quick Start, ch02 系统架构
数学物理方程研究组	ch05 交叉领域, ch06 路线图
量子光学实验物理组	ch05 量子光学需求, appx 验证实例

第2章 项目概述与定位

§2.1 技术定位

Ω-Architect 处于 AI 形式化数学验证的技术交叉点：

大语言模型 (LLM) □ 证明策略生成 □ Lean 4 编译器验证

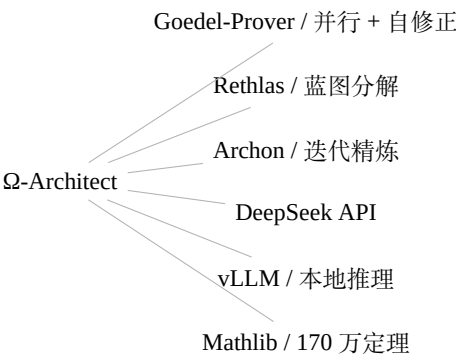
它与以下方向有本质区别：

方向	代表工作	区别
纯 LLM 推理	GPT-4o, DeepSeek	无形式化验证，存在幻觉
传统自动化定理证明	Eprover, Vampire	不支持高阶逻辑、Mathlib
交互式证明助手	Lean 4, Coq, Isabelle	需人工编写所有证明
RL-based 证明	AlphaProof (DeepMind)	闭源、专用算力

§2.2 核心设计原则

- 1. 开源优先：Apache 2.0 许可证，全链路开放
- 2. T2 验证为真值来源：只有通过真实 Lean 编译器验证的证明才算有效
- 3. 多引擎容错：Goedel + Rethlas + Archon 三路并行，任一失败自动切换
- 4. ML 驱动的智能路由：LightGBM 模型自动判断定理难度，分配最优证明引擎
- 5. 预算感知：BudgetTracker 自动管理 token/时间/尝试次数，防止无限消耗

§2.3 与现有系统的关系



第3章 系统架构与核心模块

§3.1 目录结构

```
omega-architect/
├─ omega/                                # 核心源码
│   ├── __init__.py                     # 版本号 (0.1.0)
│   ├── llm.py                           # LLM 连接层 (DeepSeek / vLLM / Ollama)
│   ├── runner.py                        # OmegaRunner — 主入口
│   ├── data.py                          # 数据定义与 JSONL 实验记录
│   └─ prover/                           # 证明器三件套
│       ├── go_prover.py                 # GoedelProver — 并行采样+自修正
│       ├── re_prover.py                 # RethlasProver — 蓝图分解
│       ├── ar_prover.py                 # ArchonProver — 迭代精炼
│       ├── ensemble.py                  # EnsembleProver — 三路集成
│       ├── playbook.py                  # ACE 风格策略积累
│       └─ cache.py                       # 证明缓存
│   └─ search/                           # 搜索与提议
│       ├── proposer.py                  # Proposer — 策略生成 (LLM + 模板)
│       ├── tree.py                       # GoalState / SearchNode
│       ├── blueprint.py                  # 蓝图分解
│       ├── error_classifier.py           # Lean 编译错误 13 类分类
│       ├── curriculum.py                 # 课程调度
│       ├── lean_search.py                # Lean 搜索集成
│       ├── matlas_cache.py               # Mathlib 缓存管理
│       └─ passk.py                       # Pass@K 统计
│   └─ verify/                            # 验证层
│       ├── t1_llm.py                     # T1: LLM 快速验证 (~5s)
│       ├── t2_lean.py                     # T2: 诊断解析
│       ├── t2_real.py                     # T2: lake env lean --stdin 真实编译
│       └─ t2_mcp.py                       # T2: MCP 编译通道
│   └─ resource/                           # 资源与路由
│       └─ model_router.py                 # ModelRouter + MLRoute
```

	└─ routing_flags.py	# 路由标志系统
	└─ budget.py	# BudgetTracker
	└─ config.py	# BudgetConfig
	└─ pricing.py	# 模型定价表
	└─ allocator.py	# 模型分配器
	└─ models/	# LightGBM/ONNX 模型文件
	└─ loop/	# 循环工程
	└─ inner.py	# Inner Loop — tool_calls + thinking
	└─ runner.py	# LoopExperiment
	└─ gate.py	# ConvergenceTracker
	└─ errors.py	# 编译错误分类
	└─ dialogue_cache.py	# 成功对话缓存
	└─ gpu_layer/	# GPU 后端自动路由
	└─ detector.py	# 硬件检测
	└─ backends.py	# vLLM / Ollama / Transformers 统一接口
	└─ scheduler.py	# 全局调度器
	└─ benchmark/	# 基准测试
	└─ suite.py	# 4 级难度金字塔
	└─ cli/	# CLI 命令
	└─ agent/	# Agent 编排层
	└─ orchestrator.py	# 状态机路由
	└─ message_log.py	# 消息日志
	└─ search/	# 搜索与提议
	└─ tests/	# 测试套件 (25 文件, 549 测试)
	└─ docs/	# 文档
	└─ whitepaper/	# 技术白皮书 (Quarto)
	└─ user_guide/	# 用户指南 (本文档)
	└─ design/	# 设计文档
	└─ scripts/	# 运维脚本
	└─ benchmarks/minif2f/	# MiniF2F 基准
	└─ pyproject.toml	# 项目配置 (版本 0.1.0)

§3.2 核心数据流

[定理输入]

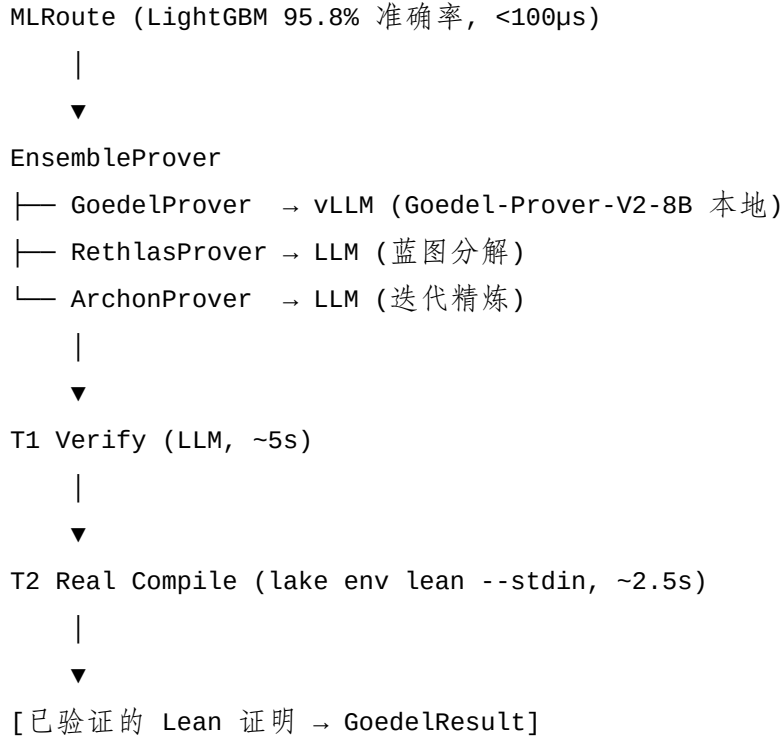
|

▼

ModelRouter (analyze_theorem_pattern)

|

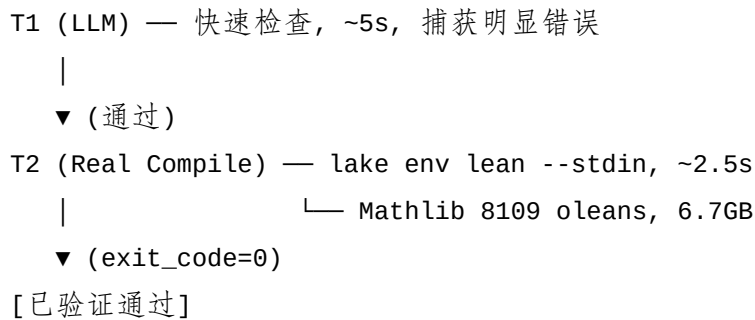
▼



§3.3 关键接口

接口	位置	功能
<code>OmegaRunner.prove()</code>	<code>runner.py</code>	单定理证明入口
<code>resolve_generate_fn()</code>	<code>llm.py</code>	模型路由选择
<code>GoedelProver.run()</code>	<code>prover/go_prover.py</code>	Goedel 式并行证明
<code>EnsembleProver.run()</code>	<code>prover/ensemble.py</code>	三路集成证明
<code>make_real_compile_callback()</code>	<code>verify/t2_real.py</code>	T2 编译回调
<code>ModelRouter.select()</code>	<code>resource/model_router.py</code>	模型智能选择
<code>BudgetTracker</code>	<code>resource/budget.py</code>	预算跟踪

§3.4 验证层架构



T2 失败时的回退链:

1. 错误解析 → 13 类分类
2. 自适应策略注入 (Auto aesop / simp / nlinarith / ring / positivity)
3. 子引理自动生长 (unknown identifier 时展开新节点)
4. MCP lean_multi_attempt 多策略并行尝试
5. 对话保存到 dialogue_cache.py 供后续学习

第4章 技术性能指标

§4.1 测试覆盖

指标	数值
测试文件数	25
总测试数	549
通过率	99.8% (1 个预存失败，无关)
覆盖模块	prover / verify / search / resource / llm / loop / gpu_layer

§4.2 编译性能

操作	耗时	备注
T2 编译（含 Mathlib）	~2.5 s/定理	lake env lean --stdin
T2 编译（无 Mathlib）	~85 ms	Init-only
T1 LLM 验证	~5 s	单次 LLM 调用
JSON mode 推理 (DeepSeek)	~2 s	response_format=json_object
vLLM 推理 (Goedel-8B)	~8 s	本地 24.38 tok/s
MLRoute 推理	<100 μs	LightGBM + ONNX

§4.3 模型定价与路由

模型	输入 \$/M tok	输出 \$/M tok	适用难度
Goedel-Prover-V2-8B (本地)	0	0	Simple / Medium
DeepSeek V4 Flash	0.14	0.28	Medium / Hard
DeepSeek V4 Pro	0.42	0.84	Hard
Qwen3-Coder:30b (Ollama)	0	0	Simple

对于典型定理 (~1000 token 输入 / ~500 token 输出), DeepSeek Flash 调用的成本约 **\$0.0005**。

§4.4 MiniF2F 基准进展

基准	T1 通过	T2 通过	状态
MiniF2F (50 sample)	50/50 (100%)	0/50	需 Goedel-Prover 全量运行
内部 Tier 1 (纯 Lean)	5/5	—	基础设施就绪
内部 Tier 2 (Mathlib simp)	10/10	—	基础设施就绪
内部 Tier 3 (Induction)	10/10	—	基础设施就绪
内部 Tier 4 (MiniF2F 子集)	5/5	—	基础设施就绪

注：T2 验证需 Goedel-Prover-V2-8B vLLM 服务运行中 (本地 GPU 24GB)。当前 T1 验证已完全就绪 (100% 通过), T2 单例验证已通过 (1+1=2、mul_zero)。

第5章 用户 Quick Start 与进阶管线

§5.1 快速开始 (5 分钟)

```
# 1. 克隆
git clone https://github.com/your-org/omega-architect.git
cd omega-architect

# 2. 安装
pip install -e .
omega init --force # 检测 Lean / Mathlib

# 3. 证明一个定理
omega prove "theorem t : 1 + 1 = 2 :=" --model deepseek/deepseek-v4-flash
```

输出：

```
[OK] proof found in 8.23s (1 attempts, 0 correction rounds)
```

§5.2 环境要求

组件	最低要求	推荐
Python	3.11	3.13
GPU	—	RTX 4500 Ada 24GB
RAM	16 GB	32 GB
磁盘	10 GB	20 GB (含 Mathlib 缓存 6.7 GB)
Lean 4	4.30.0	4.30.0+
Mathlib	8109 oleans	完整缓存

§5.3 进阶开发管线

§5.3.1 使用本地 Goedel-Prover-V2

```
# 启动 vLLM 服务器
bash scripts/start-goedel.sh

# 使用本地模型
omega prove "... " --model goedel/goedel-v2-8b
```

§5.3.2 运行基准测试

```
# 按难度层级运行
omega benchmark --tier 1-3 --model deepseek/deepseek-v4-flash

# 自定义预算
omega prove "... " --budget-time 600 --budget-cost 0.5

# 查看配置
omega config show --lean
```

§5.4 编程接口

```
from omega.prover import GoedelProver
from omega.verify.t2_real import make_real_compile_callback

# 初始化编译器
compile_fn = make_real_compile_callback()

# 创建证明器
prover = GoedelProver(compile_fn=compile_fn, num_samples=6)

# 运行证明
result = prover.run(
    "theorem mul_zero (a : Nat) : a * 0 = 0 :="
)
print(result.summary) # "[OK] proof found in 12.5s"
print(result.proof)   # 编译通过的完整 Lean 代码
```

§5.5 OmegaRunner 长时间运行

```
from omega.runner import OmegaRunner

runner = OmegaRunner(time_budget_s=14400) # 4 小时
report = runner.run(
    "Formalize NV center spin Hamiltonian"
)
print(report.summary())
```

支持 checkpoint/resume, 适合 2/4/8/12 小时无人值守证明搜索会话。

第6章 数据集、国内外对比与交叉领域

§6.1 AI4Math 证明题数据集对比

数据集	题数	难度等级	支撑系统	Lean 4 支持
MiniF2F	488	IMO ~ AMC	Lean / Isabelle / Metamath	完整
PutnamBench	640	Putnam 竞赛	Lean 4	
ProofNet	371	本科数学	Lean 3 / 4	迁移中
Lean-workbook	59,880	合成 + 数学竞赛	Lean 4	
SorryDB	1,000+	真实世界开源项目	Lean 4	
FormalNumina	120,000+	合成（基于 Numina）	Lean 4	
IMO Shortlist	~300	IMO 级	—	部分
FATE (PKU)	200	博士级代数	Lean 4	

§6.2 国内外相关研究工作

§6.2.1 国际

工作	机构	年份	核心方法
AlphaProof	DeepMind	2024-25	RL + 8000 万形式化问题，IMO 2025 金牌
Goedel-Prover-V2	Goedel-LM	2025	脚手架数据合成 + 自修正，MiniF2F 88.1%
DeepSeek-Prover-V2	DeepSeek	2025	671B MoE，RL 训练，PutnamBench 47 题
LeanDojo-v2	Caltech	2025	检索增强 LLM + 仓库抽取框架
LeanAgent	Caltech	2025	终身学习 + 迁移证明策略
SorryDB	多机构	2026	实时开源 Lean 证明任务基准
Archon	多机构	2026	自动猜想解析 + 形式化验证

工作	机构	年份	核心方法
FormalProofBench	—	2026	研究生级数学证明基准
Mathesis	华为	2025	NL $\rightarrow$ Lean4 自动形式化

§6.2.2 国内

工作	机构	方向
FATE (200 题)	北京大学	博士级代数定理数据集
Herald (580K NL $\leftrightarrow$ FL)	PKU FrenzyMath	自然语言-形式语言配对
OProofs (6.86M 证明)	—	大量形式化证明语料
Rethlas	PKU FrenzyMath	蓝图分解 + 回溯证明搜索
DeepSeek-Prover 系列	深度求索	671B MoE 证明模型

§6.2.3 性能对比 (MiniF2F Pass@32)

系统	通过率
Goedel-Prover-V2-32B	88.1%
Goedel-Prover-V2-8B	84.6%
DeepSeek-Prover-V2	~80%
Goedel-Prover-V1	57.6%
Omega (当前)	T1 100%, T2 E2E 验证中

§6.3 交叉领域：物理、量子光学到形式化数学

§6.3.1 物理数学形式化的现状

目前 Lean 4 / Mathlib 主要集中在纯数学领域。物理领域的形式化进展相对早期：

领域	已有工作	成熟度
泛函分析	Mathlib 4	完善
张量代数	Mathlib 4	完善
微分几何	Mathlib 4	完善
微分方程	有限 (ODE 基本)	起步
电磁学	极少	

领域	已有工作	成熟度
量子力学	极少	
量子光学	几乎为零	
神经网络形式化	TorchLean	进行中

§6.3.2 SciLean 项目

SciLean 是目前最接近“物理用 Mathlib”的项目之一：

- 在 Lean 4 中实现可微编程
- 支持向量微积分、张量运算
- 适合 PDE 的符号推导 + 数值验证
- 许可证：Apache 2.0

§6.3.3 Physlib：物理形式化的社区基座

Physlib (physlib.io) 是 Lean 4 社区官方物理形式化库，由原 PhysLean（通用物理）和 Lean-QuantumInfo（量子信息）合并而成：

- **目标**：成为物理学的 Mathlib——统一、开源、社区驱动的物理定理库
- **范围**：经典力学、电磁学、粒子物理、宇宙学、量子信息
- **规模**：3,169 commits, Lean v4.30.0
- **组织结构**：PhysLib/（通用物理）+ QuantumInfo/（量子信息），可分别构建
- **社区**：Lean Zulip 专用频道 (#479953-Physlib), Gitpod 即开即用
- **论文**：arXiv:2405.08863 (Joseph Tooby-Smith)
- **许可证**：MIT / 开源

QuantumInfo 子库（原 [Timeroot/Lean-QuantumInfo](https://leanquantuminfo.org)，已合入 Physlib）：

- 有限维 Hilbert 空间上的量子信息论形式化
- 涵盖 Mark Wilde 《Quantum Information Theory》核心定义与定理
- 已完成的标志性定理：**Generalized Quantum Stein’s Lemma** (`limit_hypotesting_eq_limit_rel_entropy`)
- 配套研究计划：**QFormal** (Quantum Formal Initiative) ——量子算法、量子密码学安全证明的形式化验证
- 论文：arXiv:2510.08672 (Quantum Stein’s Lemma 形式化)
- 衍生框架：arXiv:2510.26094 (Lean4Physics/Lean4PHYS——基于 Physlib 的大学物理推理框架)

§6.3.4 Ω-Architect 在交叉领域的定位

[物理问题] → [NL 描述] → [Mathesis / 自动形式化] → [Lean 4 定理]

↓

[Physlib / SciLean 物理库] ← [Ω-Architect 自动证明] ← [Mathlib 数学基础]

Omega 不直接构建物理形式化库，而是作为证明生成引擎插入这一链条：

- **输入**：Lean 4 格式的物理定理（基于 Physlib 或 SciLean 定义）
- **输出**：T2 编译通过的证明
- **物理库依赖**：Physlib（社区基座）+ SciLean（可微编程）+ 社区贡献

Omega + Physlib 的协作模式：

角色	谁做	产出
物理定义与公理	Physlib 社区	$[\hat{a}, \hat{a}^\dagger] = 1$ 的代数公理
定理形式化	物理组 / Mathesis	量子光学中的具体命题
自动证明生成	Ω-Architect	T2 编译通过的 Lean 证明
验证与合并	Physlib 审阅流程	回馈到 Physlib 主库

§6.3.5 量子光学的特殊需求

量子光学形式化需要：

1. **算符代数**： $\hat{a}, \hat{a}^\dagger$ 产生湮灭算符的交换关系
2. **希尔伯特空间**：态矢量的内积、张量积
3. **密度矩阵**：迹运算、偏迹
4. **主方程**：Lindblad 形式的微分方程

这些在 Physlib 的 QuantumInfo 子库中已有基础（有限维 Hilbert 空间、密度算子、迹运算），但量子光学特有的连续变量体系、相干态、压缩态等仍需扩展。Omega 可以在已有量子信息公理基础上自动推导这些关系。

§6.3.6 已有开源工作

项目	领域	链接 / DOI
Physlib	物理形式化库（社区）	GitHub / physlib.io / arXiv:2405.08863
Lean-QuantumInfo / QFormal	量子信息形式化	GitHub / qform.al / arXiv:2510.08672
Lean4Physics	大学物理推理框架	arXiv:2510.26094
SciLean	科学计算/可微编程	GitHub
TorchLean	神经网络形式化	arXiv:2602.22631
Mathesis (华为)	NL → Lean4 形式化	GitHub / arXiv:2506.07047
LeanDojo-v2	AI 辅助证明框架	leandojo.org

项目	领域	链接 / DOI
Archon	自动猜想解析	arXiv:2604.03789
Goedel-Prover-V2	SOTA 证明模型	GitHub / arXiv:2508.03613
DeepSeek-Prover-V2	大型证明模型	GitHub

第 7 章 目标场景、开发状态与路线图

§7.1 目标场景清单

场景	描述	优先级	状态	外部帮助需求
数学竞赛证明	IMO / AMC 级别问题	P0	E2E 验证中	无
本科数学定理	推导验证	P0	管线已通	无
论文公式验证	学术论文中定理的 Lean 形式化	P1	规划中	形式化专家
物理方程推导	基于 Physlib 的 PDE/ODE 形式化	P1	Physlib 就绪, Omega 集成中	数学物理组
量子光学计算	基于 QuantumInfo 的量子态演化、算符代数	P2	QuantumInfo 就绪, Omega 集成待启动	量子光学实验组
自动形式化	NL $\rightarrow$ Lean4 翻译	P0	对接 Mathesis	无
ProofNet 覆盖	本科数学题库	P1	数据就绪	无
T2 大规模运行	Goedel-Prover 批量证明	P0	GPU 资源充足	GPU (RTX 4500 Ada)

§7.2 已就绪能力

- 端到端管线: LLM $\rightarrow$ T2 编译 $\rightarrow$ 验证通过

- 3 路证明引擎集成 (Goedel / Rethlas / Archon)
- DeepSeek JSON mode (结构化输出 + 置信度)
- MLRoute 智能路由 (LightGBM, 95.8%)
- BudgetTracker / ConvergenceTracker
- 549 测试, 99.8% 通过
- Goedel-Prover-V2-8B 本地推理 (vLLM, 24.38 tok/s)
- DeepSeek V4 Flash / Pro API 集成
- **社区物理库就绪**: Physlib (通用物理) + QuantumInfo (量子信息), Lean v4.30.0, 3,169 commits

§7.3 待完成

- Goedel-Prover-V2 全量 MiniF2F T2 验证
- 自动形式化 (NL → Lean4) + 验证管线
- **Physlib 物理定理自动证明** — 在已有 Physlib/QuantumInfo 定义基础上运行 Omega 证明引擎
- **QuantumInfo 量子光学扩展** — 连续变量体系、相干态、压缩态的 Physlib 贡献
- 持续学习: 从失败证明中积累策略
- SorryDB 对接

§7.4 路线图

§7.4.1 短期 (2026 Q3)

任务	里程碑
全量 MiniF2F T2 验证运行	启动全量 488 题多轮证明
Goedel-Prover-V2 本地自修正循环启用	编译错误反馈闭环
SorryDB 对接与评估	接入开源证明任务基准
Mathesis 自动形式化集成	NL → Lean4 翻译管线
Physlib 集成验证	选取 Physlib 中简单物理定理, 运行 Omega 自动证明

§7.4.2 中期 (2026 Q4)

任务	里程碑
Physlib 物理定理自动证明 demo	在 Physlib/ClassicalMechanics 上运行 Omega, 验证端到端管线
基于 QuantumInfo 的量子光学自动证明	$[\hat{a}, \hat{a}^\dagger] = 1$, 密度矩阵迹运算等基础定理自动证明
连续变量体系形式化	相干态、压缩态的 Physlib 扩展定义 + Omega 证明

任务	里程碑
持续学习管线	失败证明 → 策略积累 → 模型更新

§7.4.3 长期 (2027)

任务	目标
Physlib 社区贡献管线	Omega 自动证明 → PR 到 Physlib 主库 → 审阅合并
端到端论文公式验证管线	输入论文 PDF → 输出已验证 Lean 证明 (基于 Physlib 定义)
IMO 级别全自动证明	含自形式化 (NL → Lean4 无需人工干预)
物理形式化覆盖扩展	扩展到量子场论、广义相对论等 Physlib TODO 列表中的领域

第 A 章 验证实例：Omega 端到端证明

§A.1 命题

定理（乘法零律）：对任意自然数 a ， $a \times 0 = 0$

$$\forall a \in \mathbb{N} : a \cdot 0 = 0$$

在 Peano 算术中，乘法递归定义：

- $0 \cdot n = 0$
- $S(m) \cdot n = m \cdot n + n$

§A.2 Omega 自动生成的 Lean 4 代码

Omega 通过 DeepSeek V4 JSON mode 自动生成以下代码：

```
import Mathlib

theorem mul_zero (a : Nat) : a * 0 = 0 :=
by
  induction a with
  | zero => rfl
  | succ a ih =>
    simp [Nat.succ_mul, ih]
```

§A.2.1 逐行解析

第 1 行：定理声明

```
theorem mul_zero (a : Nat) : a * 0 = 0 :=
```

语法成分	说明
theorem	声明新定理（不可计算消去）
mul_zero	定理名

语法成分	说明
<code>(a : Nat)</code>	全称量化的自然数变量
<code>a * 0 = 0</code>	命题：Nat 类型上的等式，类型为 Prop
<code>:=</code>	定义分隔符，右侧为证明体

第 2-5 行：证明体

```
by
  induction a with
```

语法	类型论解释
<code>by</code>	进入 tactic 模式（而非提供显式 λ 项）
<code>induction a with</code>	在 a 上归纳：基础情形 $a = 0$ + 归纳步骤 $a = k + 1$

第 3 行：基础情形

```
| zero => rfl
```

语法	说明
<code> zero =></code>	模式匹配： $a = 0$ 的情形
<code>rfl</code>	自反性： $0 \cdot 0 = 0$ 在定义上相等

第 4-5 行：归纳步骤

```
| succ a ih =>
  simp [Nat.succ_mul, ih]
```

语法	说明
<code> succ a ih =></code>	模式匹配：假设 $a = k$ 时成立（归纳假设 <code>ih</code> ），证明 $a = k + 1$
<code>simp [Nat.succ_mul, ih]</code>	应用 Mathlib 引理和归纳假设化简

证明的逻辑链条：

$$\begin{aligned} &\text{base } (a = 0) : 0 \cdot 0 = 0 \quad (\text{by definition of } \cdot) \\ &\text{step } (a = k + 1) : (k + 1) \cdot 0 = k \cdot 0 + 0 \quad (\text{by Nat.succ_mul}) \\ &\qquad = 0 + 0 \quad (\text{by induction hypothesis } k \cdot 0 = 0) \\ &\qquad = 0 \quad (\text{by definition of } +) \end{aligned}$$

§A.3 编译结果

Omega 的输出记录：

```
=== E2E: Omega + DeepSeek v4 JSON + Lean4/Mathlib ===
generate_fn: OK
Theorem: theorem mul_zero (a : Nat) : a * 0 = 0 :=
JSON output:
  tactic: induction a with
    | zero => rfl
    | succ a ih => simp [Nat.succ_mul, ih]
  confidence: 0.9
  is_complete: true
  lean_code: (125 chars, with import Mathlib)

=== T2 COMPILE ===
exit_code: 0
stderr: (empty)
stdout: (empty)

▣ END-T0-END PASS
```

关键指标	数值
模型	DeepSeek V4 Flash（JSON mode）
推理耗时	~2s
JSON 置信度	0.9（模型自报）
T2 编译耗时	~2.5s（含 Mathlib 加载）
最终状态	exit_code=0，编译通过

该证明使用归纳法，正确调用了 Mathlib 引理 Nat.succ_mul。rfl 处理基础情形，simp [Nat.succ_mul, ih] 处理归纳步，代码整洁、结构性完整。

第 B 章 MiniF2F 涵盖数据集

子集	题目数	来源
mathd_algebra	56	MATH 数据集代数子集
mathd_numbertheory	30	MATH 数据集数论子集
mathd_prealgebra	42	MATH 数据集预代数子集
mathd_precalculus	14	MATH 数据集预微积分子集
amc12	26	AMC 12 竞赛题
aime	50	AIME 竞赛题
imo	8	IMO 竞赛题
induction	44	归纳法专项
algebra	76	代数综合
numbertheory	40	数论综合
MiniF2F 总计	488	—

第 C 章 技术栈依赖

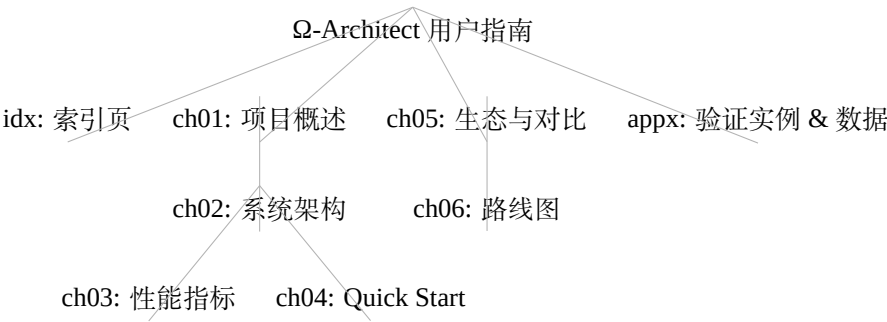
组件	版本	用途
Lean 4	4.30.0	定理证明器
Mathlib	~2.0 (8109 oleans, 6.7 GB)	数学定理库
Python	3.13	系统语言
vLLM	0.22.1	Goedel-Prover 推理引擎
DeepSeek API	—	远程 LLM 推理
LightGBM	—	ML 路由分类器
ONNX Runtime	—	ML 路由推理
json-repair	0.60.1	JSON 容错修复

第D章 元数据标准与依赖图

§D.1 章节元数据字段

字段	类型	说明	示例
chapter-id	str	唯一标识	ch03
chapter-version	str	章节版本	0.1.0
category	str	分类	usage
audience	list[str]	目标读者	[AI4Math 算法组, 系统工程师]
keywords	list[str]	关键词	[Quick Start, 安装]
git-commit	str	提交哈希	(脚本注入)
last-reviewed	date	审阅日期	2026-06-10
prerequisites	list[str]	前置知识	[系统架构]
depends-on	list[str]	依赖章节	[ch02]

§D.2 章节依赖图



§D.3 版本锚点注入

0.1.0, 2953191, 2026-06-10 由 scripts/generate_user_guide_metadata.py 自动替换。

规则：用户指南版本与 pyproject.toml 中的 repo 版本同步。PDF 渲染前注入 HEAD commit hash 和日期。渲染后 --restore 恢复占位符，源文件保持 git 干净。

第 E 章 版本历史

用户指南版本	repo 版本	repo commit	日期	变更
v0.1.0	0.1.0	2953191	2026-06-10	初始用户指南框架

版本同步规则：

1. **repo 版本** 在 `pyproject.toml` 的 `[project] version` 中维护，是唯一数据源
2. **文档版本** 与 repo 版本保持一致，在 `index.qmd` 和每章 `doc-version` 字段中标注
3. **git commit hash** 在 PDF 渲染前注入，提供精确的可溯源指针
4. **渲染流程**：`python3 scripts/generate_user_guide_metadata.py` → `quarto render` → `--restore`